Getting Started with GitHub and Finding Good Qiskit First Issues

A hands-on workshop for QGSS 2026

Steven Hergenrother and James Weaver: IBM Quantum : QGSS 2026 GitHub Workshop

Agenda

1. Welcome and Introduction
2. Setting Up Your GitHub Account
3. Understanding Repositories
4. Basic Git Workflow on GitHub
5. Collaboration with Pull Requests
6. Fork and Local Workflow Overview
7. Finding Good First Issues in Qiskit
8. Further Learning and Next Steps
9. Wrap-Up and Q&A

Welcome and Introduction

Quick Poll

How much experience do you have with Git and GitHub?

- Never used it
- Created an account but haven't done much
- Used it a few times
- Use it regularly

What Is GitHub?

GitHub is a platform where people **store work**, **track changes**, and **collaborate**.

- **Version control** -- every change is recorded and reversible
- **Collaboration** -- teams and communities work together on code and documents
- **Open source home** -- millions of projects, including Qiskit, live here

> Think of GitHub as Google Docs for code: everyone can see the history, suggest changes, and work together.

What You Will Be Able to Do After This Workshop

- ☑ Create a GitHub repository
- ☑ Edit files and commit changes in the browser
- ☑ Create a branch and open a pull request
- ☑ Find beginner-friendly Qiskit issues to contribute to
- ☑ Understand the fork-and-clone workflow for contributing to open source

Setting Up Your GitHub Account

Create or Sign In to Your Account

1. Go to github.com (https://github.com)
2. **New users:** Click **Sign up** and follow the prompts
3. **Existing users:** Click **Sign in**

Key settings to check

- **Display name** and **profile picture** -- helps others identify you
- **Email** -- make sure it is verified
- **Notifications** -- default settings are fine for now

The GitHub Dashboard

Once signed in, locate these key areas:

ELEMENT	WHAT IT DOES
+ **menu** (top right)	Create new repos, import projects
Profile menu (avatar)	Access your profile, settings, repos
Repositories list	Your projects appear here
Search bar	Find repos, users, and code across GitHub

> Action: Verify you can see the + menu and your profile menu.

Understanding Repositories

What Is a Repository?

A **repository** (or **repo**) is a project folder that contains:

- Your **files** (code, documents, images)
- The complete **history** of every change
- **Collaboration features** (issues, pull requests, discussions)

> A repo is the basic unit of work on GitHub. Everything starts here.

Creating Your First Repository

1. Click the **+** menu in the top-right corner
2. Select **New repository**
3. Fill in the details:

FIELD	WHAT TO ENTER
Repository name	`my-first-repo` (short, descriptive, lowercase)
Description	A brief summary of the project
Visibility	**Public** (anyone can see) or **Private** (only you)
Initialize with README	Check this box

4. Click **Create repository**

Exploring the Repository Interface

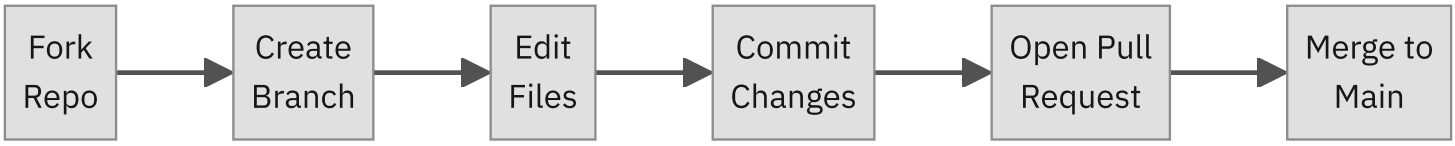
Key elements on a repository page

- **Code tab** -- your files and folders
- **File browser** -- click any file to view it
- **README** -- displays automatically below the file list
- **Edit button** (pencil icon) -- edit files directly in the browser

> The README is the front page of your project. It tells visitors what the project is about.

Basic Git Workflow on GitHub

The Core Workflow



Today we will practice the middle steps on your own repo in the browser. Later, we will see how **forking** adds the first step when contributing to someone else's project.

Editing Files in the Browser

1. Navigate to a file in your repo (e.g., `README.md`)
2. Click the **pencil icon** to edit
3. Make your changes in the editor
4. Scroll down to the **Commit changes** section

> No software to install. No command line. Just click, type, and save.

Understanding Commits

A **commit** is a saved snapshot of your changes, with a message explaining what you did.

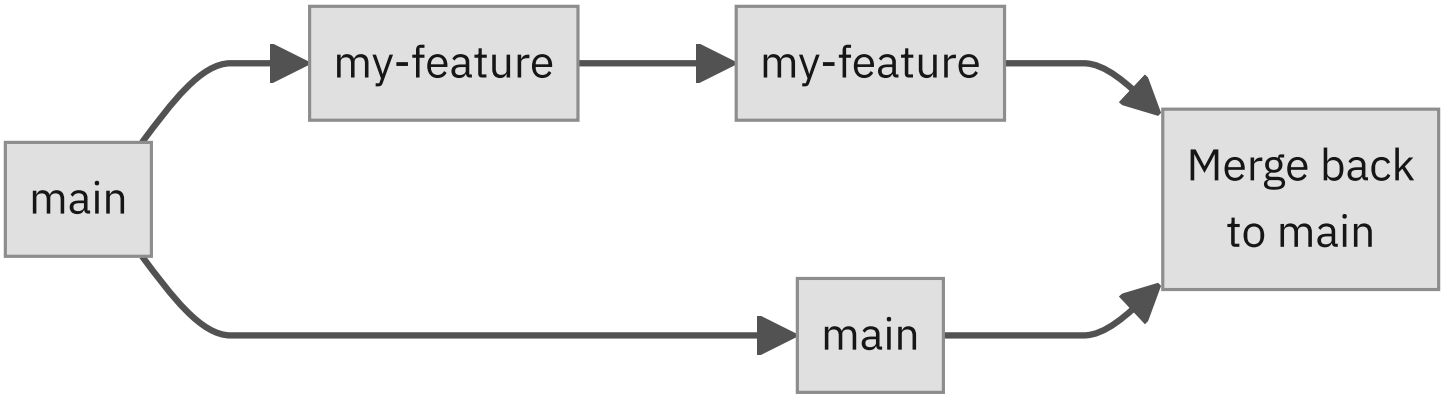
Good vs. vague commit messages

MESSAGE	VERDICT
`Update README.md`	Vague -- what was updated?
`Add project description to README`	Clear -- explains the change

> Write commit messages that tell *what* changed and *why*. Your future self (and collaborators) will thank you.

What Are Branches?

A **branch** is a parallel copy of your project where you can make changes without affecting the main version.



- `main` -- the primary branch; the "official" version
- **Feature branches** -- safe spaces to try changes before merging

> Branches let you experiment without risk. If something goes wrong, `main` is untouched.

Creating a Branch

1. Go to your repository page
2. Click the **branch dropdown** (says `main`)
3. Type a new branch name (e.g., `add-about-page`)
4. Click **Create branch: add-about-page from main**

You are now working on your new branch. Changes here will not affect `main`.

Hands-On Exercise: Branch, Edit, Commit

Try it yourself:

1. Create a new branch called `add-about-page`
2. On that branch, create a new file called `about.md`
   - Click **Add file** then **Create new file**
   - Name it `about.md`
   - Add a line: `This is my QGSS practice project.`
3. Write a clear commit message and commit the file

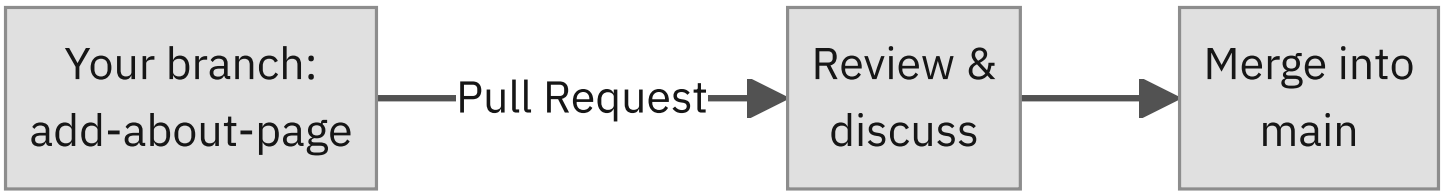
Success check

- ☐ You have a branch that is NOT `main`
- ☐ That branch has a new file with one commit

Collaboration with Pull Requests

What Is a Pull Request?

A **pull request** (PR) is a proposal to merge changes from one branch into another.



- The name means: *"Please pull my changes into* `main` *."*
- It is the **review-and-merge step** in a collaborative workflow
- Others can view your changes, leave comments, and approve

> A pull request says: "Here is what I changed, here is why, and I'd like it merged."

Creating a Pull Request

1. After committing to your branch, GitHub shows a **Compare & pull request** banner — click it
2. Or go to the **Pull requests** tab and click **New pull request**
3. Fill in the details:

FIELD	WHAT TO ENTER
Base branch	`main` (where changes go)
Compare branch	`add-about-page` (your changes)
Title	Short summary of the change
Description	What you changed and why

4. Click **Create pull request**

Reviewing and Merging

Reviewing changes

- The **Files changed** tab shows the **diff** — what was added (green) and removed (red)
- Team members can leave **comments** on specific lines

Merging

- Once the changes look good, click **Merge pull request**
- Then click **Confirm merge**
- Your changes are now part of `main`

> After merging, you can safely delete the feature branch — its work is in `main` now.

Hands-On Exercise: Open and Merge a PR

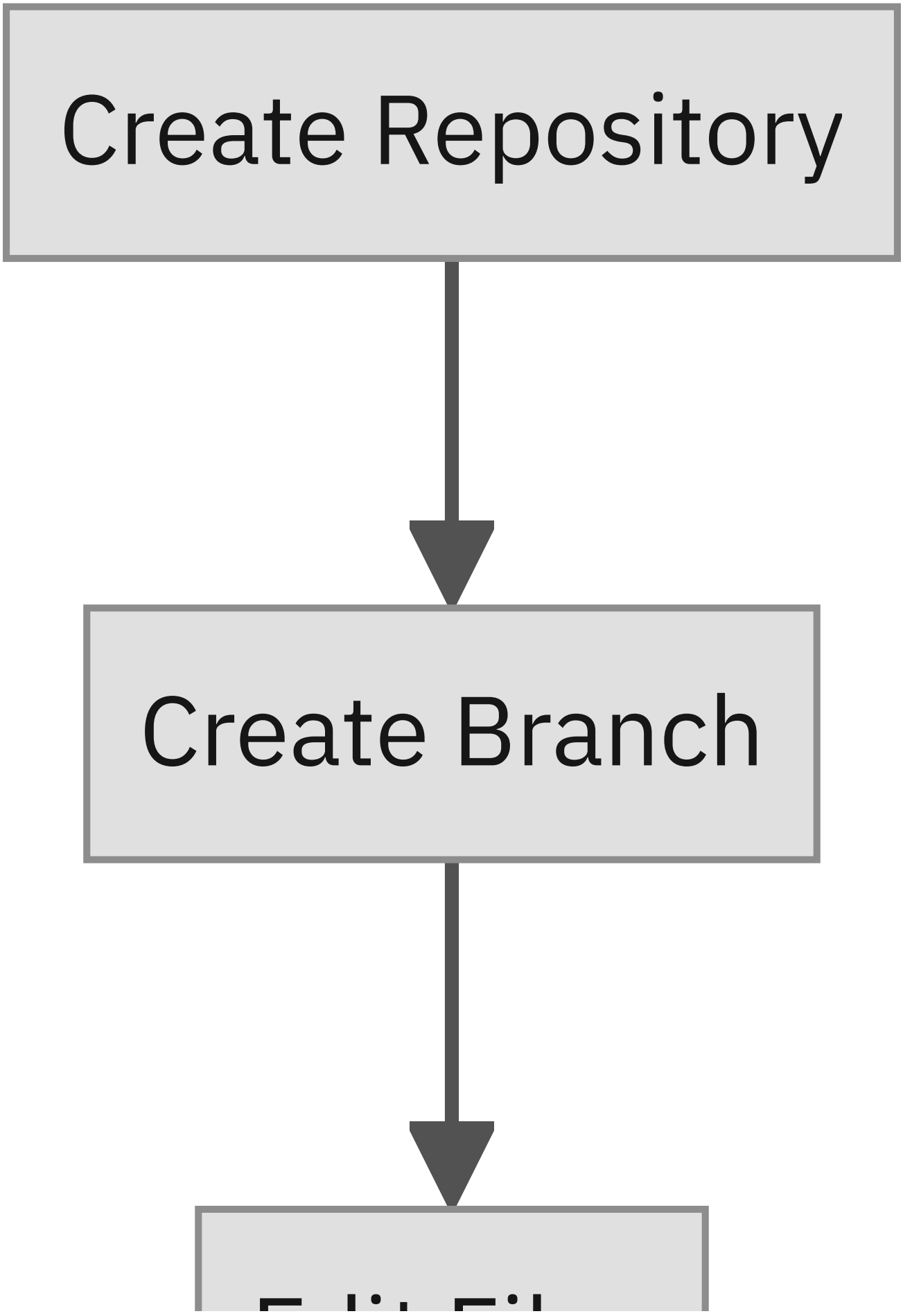
Try it yourself:

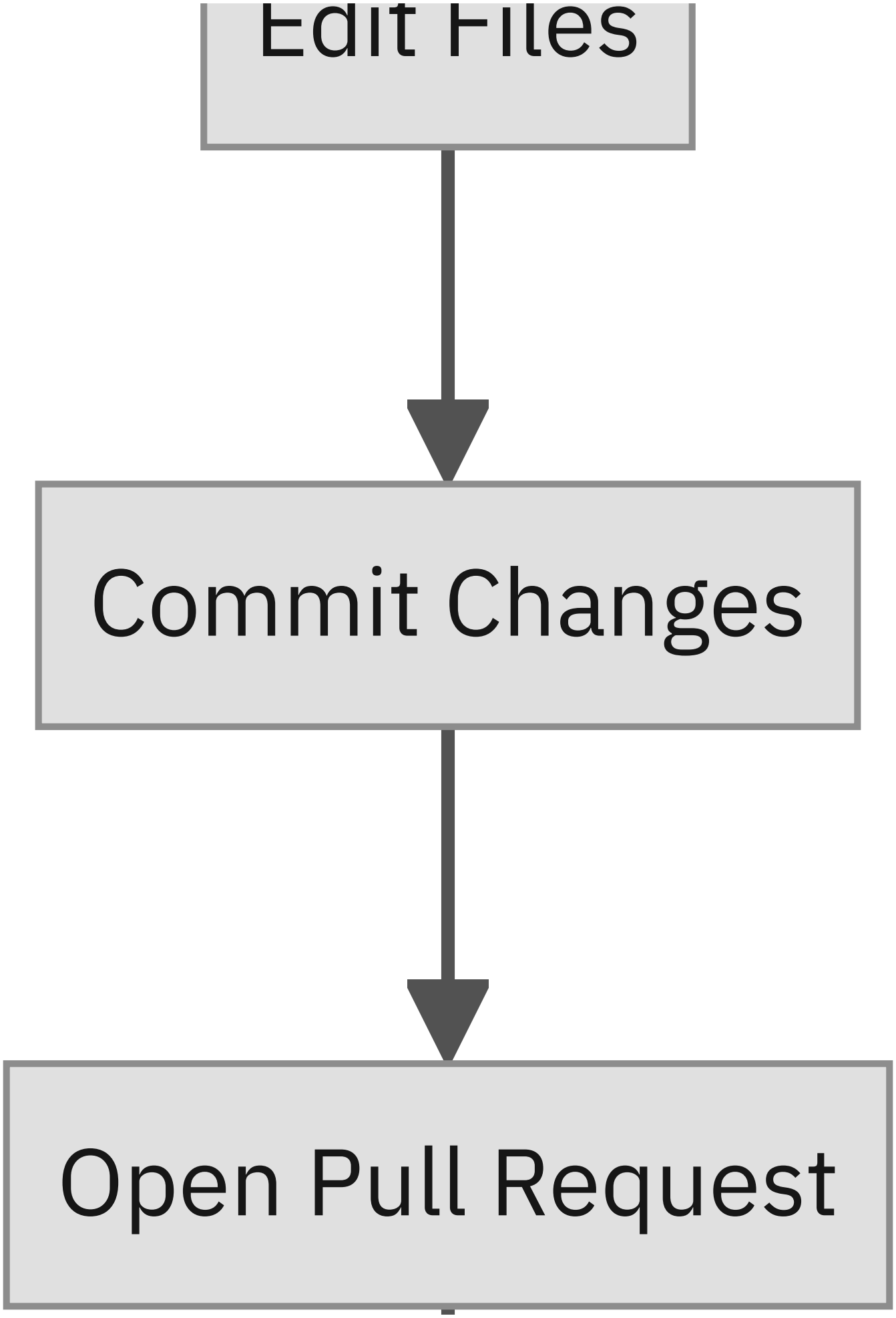
1. Open a pull request from `add-about-page` into `main`
2. Look at the **Files changed** tab to see your diff
3. Click **Merge pull request**, then **Confirm merge**

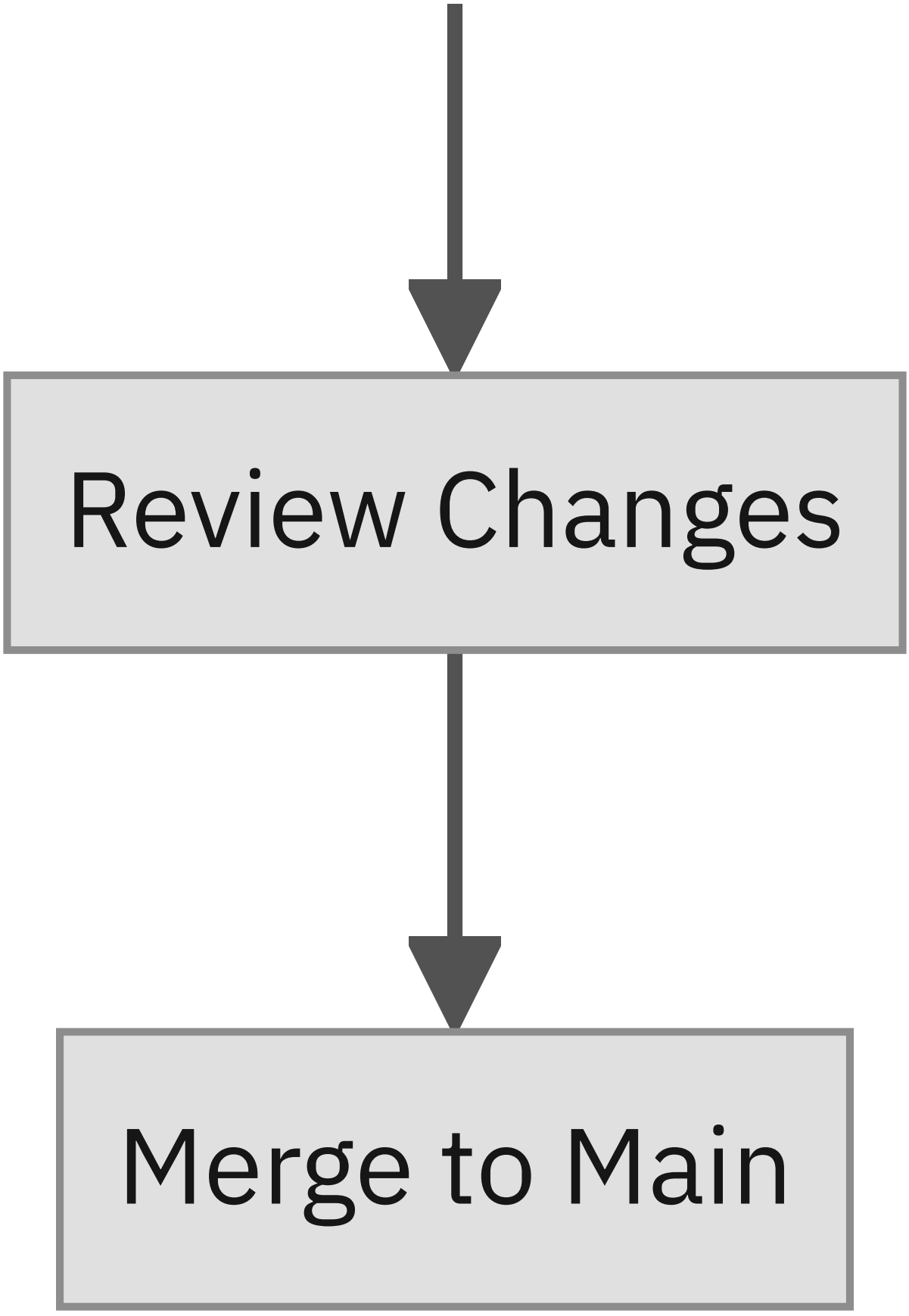
Success check

- ☐ You opened a pull request
- ☐ You can see the diff showing your changes
- ☐ You merged the PR into `main`

The Complete Workflow


Create Repository
Create Branch


Edit Files
Commit Changes
Open Pull Request



You just completed the core GitHub workflow. This is the same process used by teams and

> open source projects worldwide — including Qiskit.

Fork and Local Workflow Overview

From Issue to Contribution

Soon you'll be hunting for Qiskit issues to work on. First, how do you actually contribute to a repo you don't own?

Contributing to someone else's repository requires a **fork**: your own copy of the repo on GitHub, linked back to the original.

- You **cannot** push changes directly to a repo you don't own (like Qiskit)
- A **fork** creates a personal copy under your GitHub account
- Forking is a **one-click** operation in the browser

> A fork is your personal workspace. You make changes there, then propose them back to the original project.

Forking a Repo

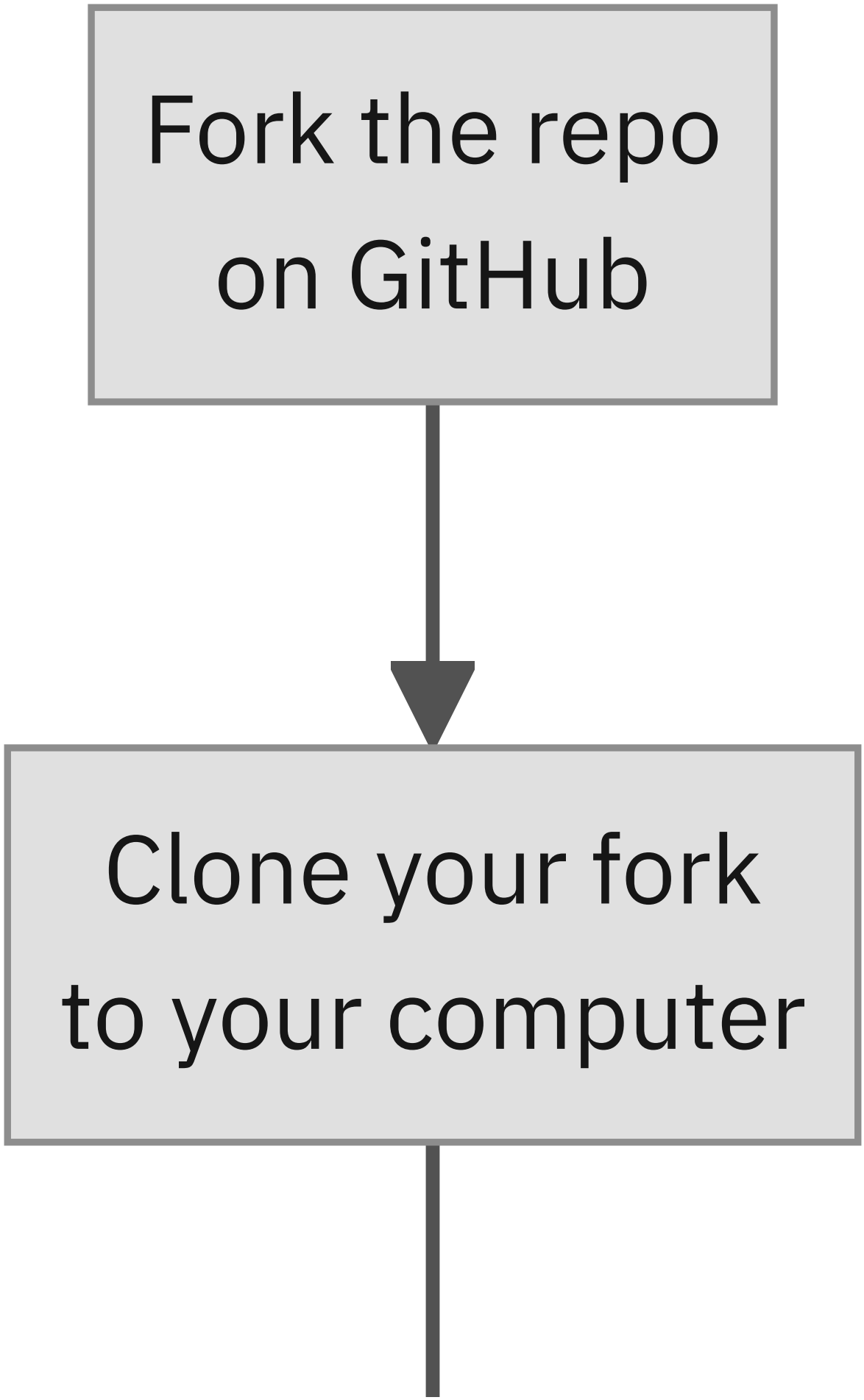
1. Navigate to the repository you want to contribute to
2. Click the **Fork** button (top right)
3. Confirm by clicking **Create fork**
4. The fork appears under your account

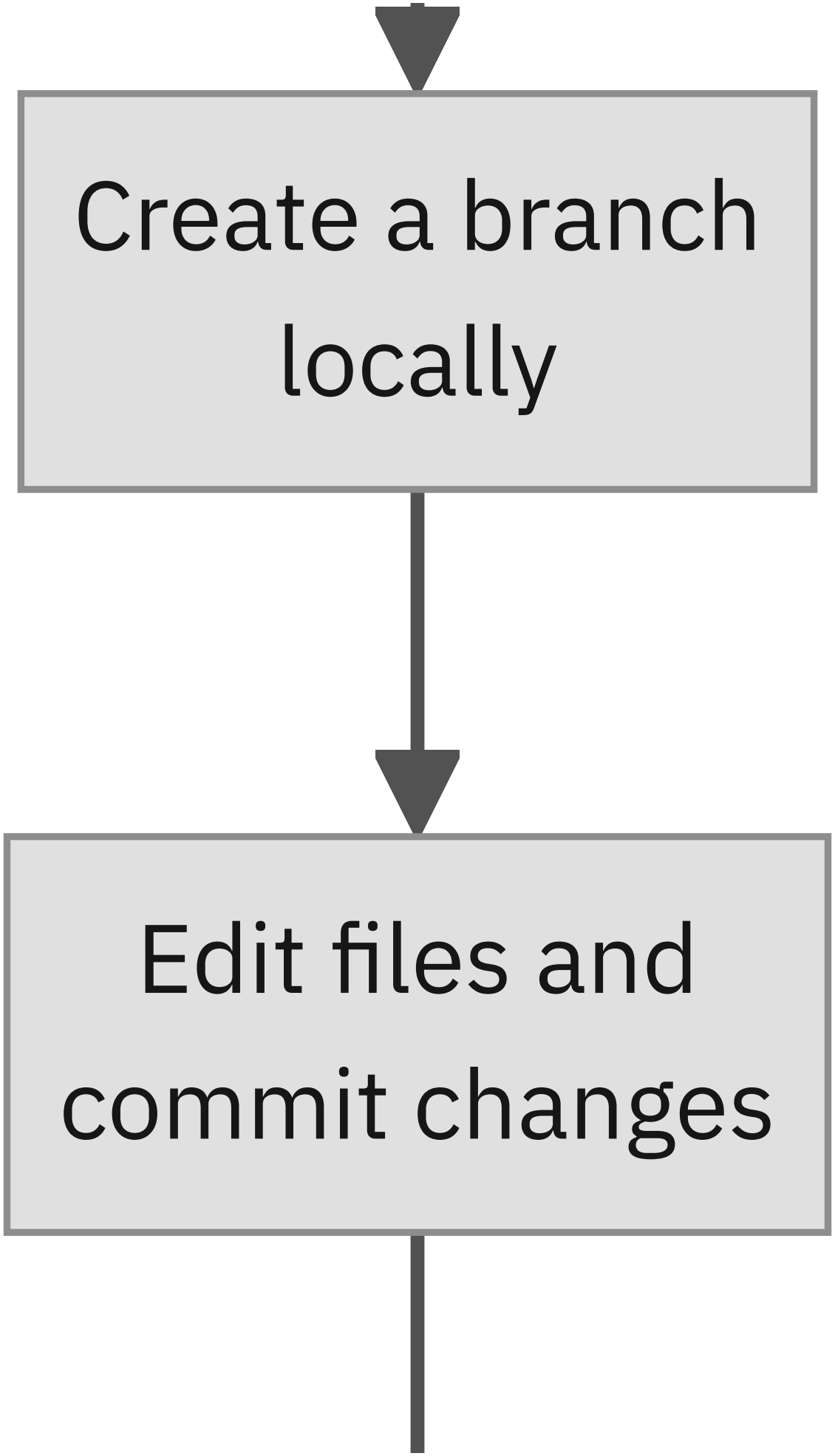
	ORIGINAL REPO	YOUR FORK
Owner	`Qiskit/documentation`	`your-username/documentation`
You can push to it	No	Yes
Linked to original	--	Yes (upstream)

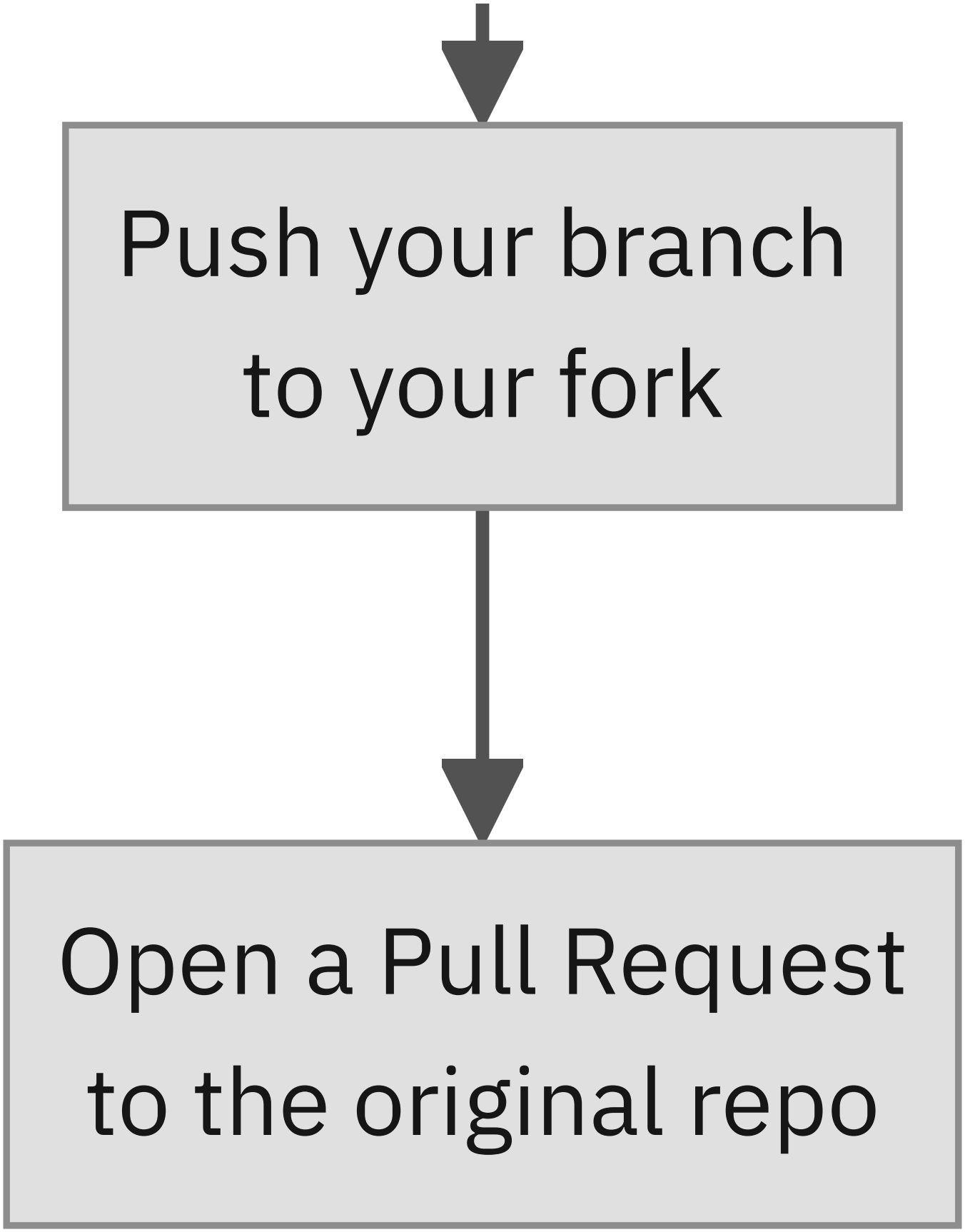
> Notice the "forked from" link at the top of your fork — it connects back to the original project.

The Local Contribution Workflow

Once you have a fork, here is the typical workflow for making a contribution:


Fork the repo on GitHub
Clone your fork to your computer


Create a branch locally
Edit files and commit changes



This is the same workflow used by Qiskit contributors worldwide. Each step is straightforward

> once you have done it once.

What Each Step Looks Like

STEP	WHAT HAPPENS	WHERE
Fork	Click **Fork** on GitHub	Browser
Clone	`git clone https://github.com/you/repo.git`	Terminal
Branch	`git checkout -b fix-typo`	Terminal
Edit	Open files in any editor and make changes	Local machine
Commit	`git add .` then `git commit -m "Fix typo in docs"`	Terminal
Push	`git push origin fix-typo`	Terminal
Open PR	Click **Compare & pull request** on GitHub	Browser

> You do not need to memorize these commands today. The key takeaway is the flow: fork, clone, branch, edit, commit, push, PR.

Fork Workflow vs. Branch Workflow

How does this compare to what you practiced earlier?

	BRANCH WORKFLOW (PRACTICED TODAY)	FORK WORKFLOW (OPEN SOURCE)
When to use	You have write access to the repo	You do NOT have write access
Where you work	Branch in the same repo	Branch in your own fork
PR target	Your branch to `main` in the same repo	Your fork's branch to `main` in the original repo
Practiced today?	Yes (hands-on)	Visual walkthrough only

> The mechanics are the same: branch, edit, commit, PR. The fork just gives you a place to do it when you don't own the repo.

Finding Good First Issues in Qiskit

What Are GitHub Issues?

Issues are how projects track work:

- **Bug reports** -- something isn't working correctly
- **Feature requests** -- ideas for new functionality
- **Tasks** -- specific work items for contributors

Anatomy of a good issue

ELEMENT	PURPOSE
Title	Clear, specific summary
Description	Details about the problem or task
Labels	Categories like `bug` , `enhancement` , `good first issue`
Assignee	Who is working on it

The "Good First Issue" Label

Maintainers add the `good first issue` label to signal:

- The task is **approachable** for newcomers
- The scope is **bounded** and well-defined
- The required context is **limited** — you don't need to understand the entire codebase

> "Good first issue" is an invitation. Maintainers *want* new contributors to pick these up.

The Qiskit Organization on GitHub

Qiskit is an open source quantum computing SDK with multiple repositories:

REPOSITORY	WHAT IT CONTAINS
qiskit (https://github.com/Qiskit/qiskit)	Core SDK — circuits, transpiler, primitives
documentation (https://github.com/Qiskit/documentation)	Qiskit docs and tutorials
And more...	Ecosystem tools and extensions

> All of these repos have issues — and many have good first issues waiting for contributors.

How to Find Good First Issues

Method 1: The contribute page

Go to github.com/Qiskit/qiskit/contribute (https://github.com/Qiskit/qiskit/contribute)

This page automatically shows issues labeled `good first issue`.

Method 2: Filter the issues list

1. Go to the **Issues** tab in any Qiskit repo
2. Use the **Labels** dropdown and select **good first issue**
3. Or type in the search bar:

```
label:"good first issue"
```

Types of Beginner-Friendly Qiskit Contributions

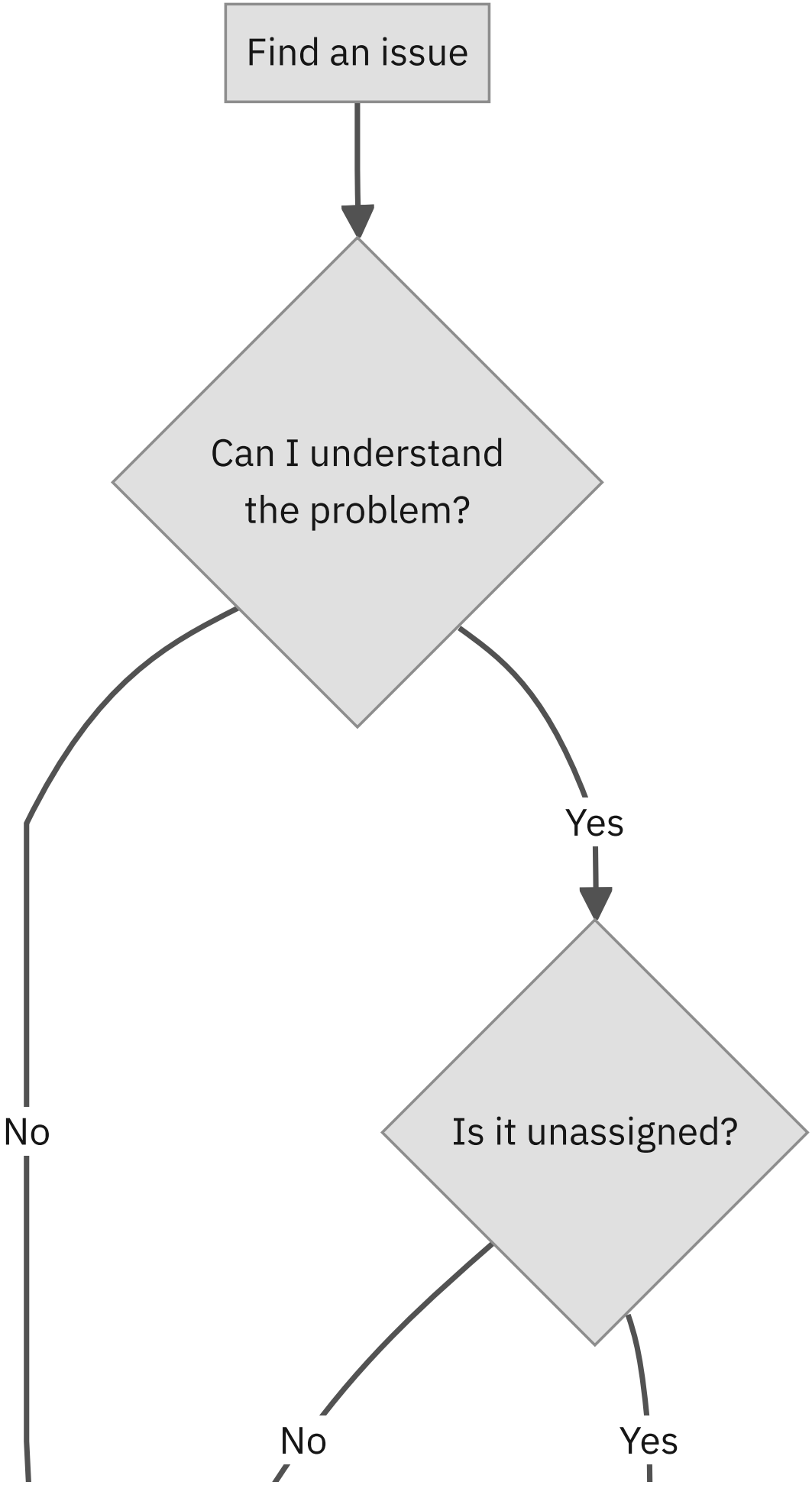
CONTRIBUTION TYPE	EXAMPLE
Documentation fixes	Fix typos, improve formatting, add examples
Bug fixes	Low-priority bugs with clear reproduction steps
Testing improvements	Add missing test cases, improve coverage
Code quality	Clean up deprecation warnings, improve error messages

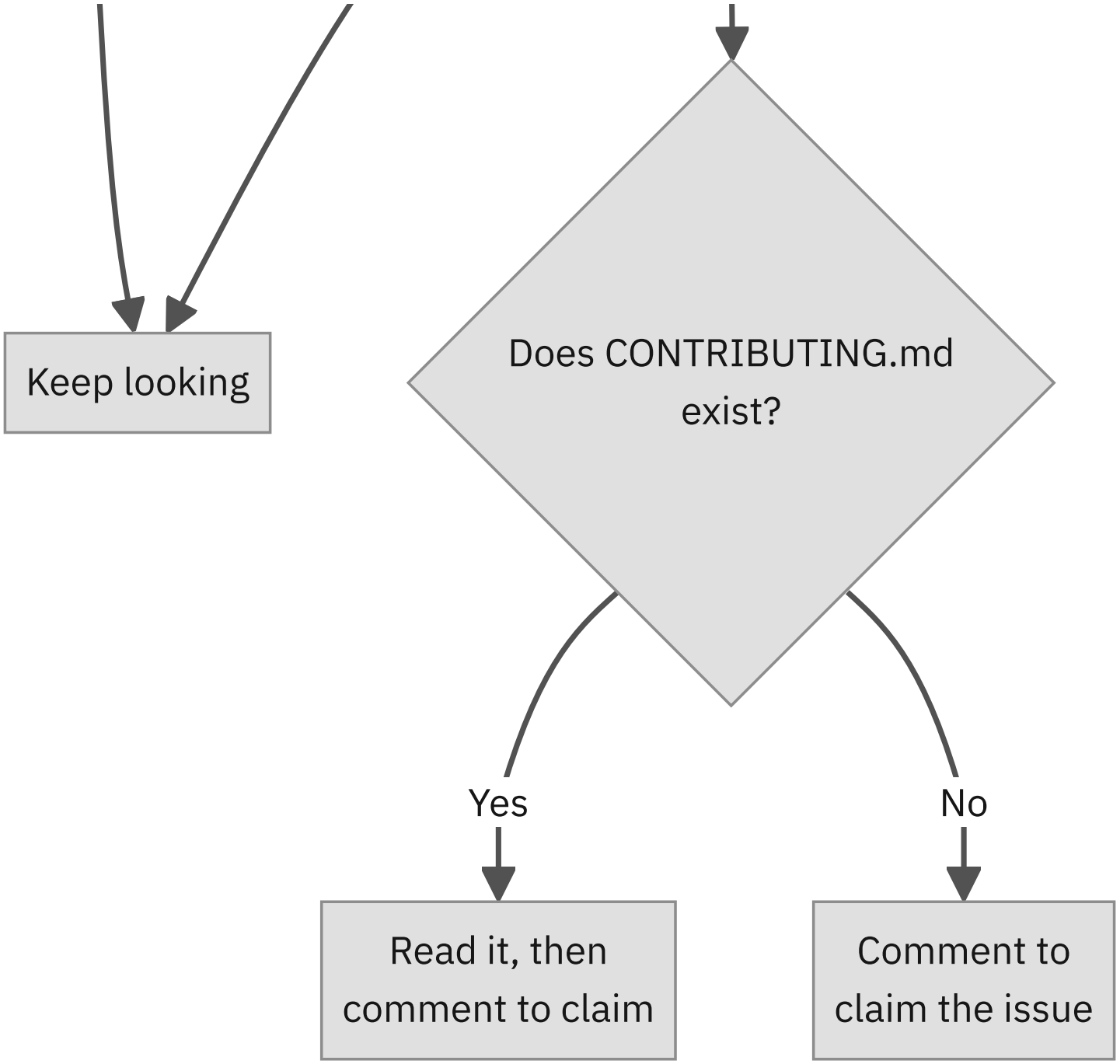
> Documentation contributions are a great entry point — even without deep quantum computing knowledge.

Evaluating an Issue

When you find an issue, ask yourself:

1. **What problem is being described?** Can I understand it?
2. **Is the task bounded?** Is it clear when the work is "done"?
3. **Is there contribution guidance?** Does the repo have a `CONTRIBUTING.md` ?
4. **Is it already assigned?** Check the assignee field


Find an issue
Can I understand the problem?
Yes
No
Is it unassigned?
No
Yes


Keep looking
Does CONTRIBUTING.md exist?
Yes
No
Read it, then comment to claim
Comment to claim the issue

Before You Start Contributing

Essential etiquette

1. **Read `CONTRIBUTING.md`** -- every repo has contribution guidelines
2. **Check assignment** -- make sure the issue isn't already claimed
3. **Comment first** -- write something like: *"I'd like to work on this. Could you assign it to me?"*
4. **Wait for confirmation** before starting work
5. **DO NOT JUST ASK AN LLM TO FIX THESE ISSUES**
6. **Keep it simple** -- the issue doesn't have to be labeled `good first issue`, but pick something small and well-scoped
7. **Contributing well is its own reward** -- learning how to contribute properly is enough learning on its own

> Don't start coding before commenting. Claiming an issue avoids duplicate work and shows respect for the community.

Live Demo: Finding a Good First Issue

Let's walk through it together:

1. Visit github.com/Qiskit/qiskit/contribute (https://github.com/Qiskit/qiskit/contribute)
2. Browse the available issues
3. Open one issue and evaluate it together
4. Also check github.com/Qiskit/documentation/issues (https://github.com/Qiskit/documentation/issues) for doc-related issues

Your task: Identify one issue that looks interesting to you. You don't need to start it now — just bookmark it for later.

Further Learning and Next Steps

Where to Go from Here

Topics for continued exploration

TOPIC	WHAT IT IS
GitHub Actions	Automate testing, building, and deployment
GitHub Pages	Host a simple website directly from a repo
Command-line Git	More powerful workflows using the terminal

> These are follow-on topics — not required for today. You already have everything you need to start contributing.

Learning Resources

- **GitHub Docs** (https://docs.github.com) -- official documentation and guides
- **GitHub Skills** (https://skills.github.com) -- hands-on interactive courses
- **Qiskit Contributing Guide** (https://github.com/Qiskit/qiskit/blob/main/CONTRIBUTING.md) -- how to contribute to Qiskit

> Bookmark these links. GitHub Skills is especially good for structured, self-paced learning.

Wrap-Up and Q&A

What We Covered Today

1. **Repositories** -- created your own project on GitHub
2. **Branches** -- worked safely without touching `main`
3. **Commits** -- saved changes with clear messages
4. **Pull Requests** -- proposed, reviewed, and merged changes
5. **Issues** -- found beginner-friendly Qiskit issues to contribute to
6. **Fork workflow** -- saw how to fork, clone, and open a PR to an upstream project

Your Next Step

> Revisit the Qiskit issue list later this week and shortlist one issue to explore more deeply.

Quick links

- Qiskit good first issues: github.com/Qiskit/qiskit/contribute (https://github.com/Qiskit/qiskit/contribute)
- Qiskit docs issues: github.com/Qiskit/documentation/issues (https://github.com/Qiskit/documentation/issues)
- GitHub Skills: skills.github.com (https://skills.github.com)

Remember: **documentation contributions are a great starting point**, even without deep quantum computing knowledge.

Questions?

> Thank you for joining the workshop!